Automaton Specification Language Documentation

Automata Simulator

Contents

1 Introduction **2**

2 General Syntax **2**
2.1 File Structure . . . 2
2.2 Lexical Rules . . . 2
2.3 State Ranges . . . 2
2.4 Special Symbols . . . 2

3 Finite Automata **3**
3.1 Deterministic Finite Automata (DFA) . . . 3
3.1.1 Required Sections . . . 3
3.1.2 DFA Transition Syntax . . . 3
3.1.3 DFA Settings . . . 3
3.1.4 Applications and Use Cases . . . 3
3.2 Non-deterministic Finite Automata (NFA) . . . 4
3.2.1 Required Sections . . . 4
3.2.2 NFA Transition Syntax . . . 4
3.2.3 Applications and Use Cases . . . 4

4 Pushdown Automata (PDA) **5**
4.0.1 PDA Specific Sections . . . 5
4.0.2 PDA Transition Syntax . . . 5
4.0.3 Applications and Use Cases . . . 5

5 Context-Free Grammars (CFG) **6**
5.0.1 Required Sections . . . 6
5.0.2 CFG Rule Syntax . . . 6
5.0.3 CFG Execution Modes . . . 6
5.0.4 Implementation Details . . . 6
5.0.5 Applications and Use Cases . . . 6

6 Automaton Transformation Tools **7**
6.1 DFA / NFA Conversion . . . 7
6.2 DFA Minimisation . . . 7
6.3 Automaton Visualization . . . 7

1 Introduction

The Automaton Specification Language (ASL) is a domain-specific language designed to define finite automata, pushdown automata, and context-free grammars for simulation. It currently supports four types:

- **DFA**: Deterministic Finite Automata
- **NFA**: Non-deterministic Finite Automata
- **PDA**: Pushdown Automata
- **CFG**: Context-Free Grammars

2 General Syntax

All ASL files share a common structure based on sections and lexical rules.

2.1 File Structure

An ASL file is divided into sections, each denoted by a header in square brackets (e.g., `[alphabet]`). Sections can appear in any order.

2.2 Lexical Rules

- **Comments**: Any text following a `#` symbol on a line is ignored.
- **Whitespace**: Leading and trailing whitespace on lines is ignored. Empty lines are ignored.
- **Case Sensitivity**: Section headers are case-insensitive (`[ALPHABET]` is the same as `[alphabet]`).

2.3 State Ranges

In sections defining states (like `[states]` or `[accept_states]`), you can use range syntax to define multiple states at once: `prefix[start-end]`.

```
q[0-5] # Expands to q0 q1 q2 q3 q4 q5
```

2.4 Special Symbols

- `*`: Wildcard representing all symbols in the defined alphabet.
- `&`: Epsilon symbol, representing an empty transition (no input consumed).

3 Finite Automata

This section groups the finite-automata formalisms supported by the project.

3.1 Deterministic Finite Automata (DFA)

DFAs are the simplest form of automata, where each state has exactly one transition for each input symbol.

3.1.1 Required Sections

- `[alphabet]`: Space-separated list of accepted symbols.
- `[states]`: List of all possible states.
- `[initial_state]`: The starting state (exactly one).
- `[accept_states]`: States that result in an 'Accepted' status.
- `[transitions]`: Rules for moving between states.

3.1.2 DFA Transition Syntax

Each transition must follow the format:

```
source_state, symbol -> next_state
```

3.1.3 DFA Settings

Optional behaviors can be enabled in the `[settings]` section:

- `RemainInCurrentStateOnUndefinedTransition`: If enabled, the automaton stays in its current state instead of hanging when a symbol has no defined transition.

3.1.4 Applications and Use Cases

DFAs are widely used in applications where predictable, state-based behavior is required. Common use cases include:

- **Lexical Analysis**: Compilers use DFAs to recognize tokens (keywords, identifiers) in source code.
- **Protocol Verification**: Ensuring that network or hardware protocols follow a strict sequence of states.
- **Game Logic**: Controlling state transitions in text-based adventures or simple NPC behaviors.

Practical Example: String Filtering The following DFA (found in `examples/dfa/only_as/`) filters strings to ensure they consist only of the character 'a'. Any 'b' character leads to a permanent "trap" state (`q2`).

```
[transitions]
q1, a -> q1
q1, b -> q2
q2, a -> q2
q2, b -> q2

[initial_state]
q1

[accept_states]
q1
```

Listing 1: DFA for matching only 'a's

Practical Example: Adventure Game The emulator's interactive mode can be used to simulate a text-based adventure game (`examples/dfa/adventure_game_basic/`). By enabling `RemainInCurrentStateOnUndefined` the game becomes more robust to invalid user commands.

```
[initial_state]
Entrance

[transitions]
Entrance, W -> Hallway
Garden, N -> Heaven
Garden, S -> Hell
Hallway, W -> Library
Hallway, N -> Kitchen
Kitchen, W -> DiningRoom
DiningRoom, W -> Garden

[settings]
RemainInCurrentStateOnUndefinedTransition
```

Listing 2: DFA for a simple adventure game

3.2 Non-deterministic Finite Automata (NFA)

NFAs allow for multiple possible next states for a given input symbol and support epsilon transitions.

3.2.1 Required Sections

NFAs use the same sections as DFAs (`[alphabet]`, `[states]`, etc.).

3.2.2 NFA Transition Syntax

NFAs support flexible syntax to handle non-determinism:

- **Multiple Targets**: `q0, a -> q1, q2`
- **Multiple Symbols**: `q0, (a, b) -> q1`
- **Epsilon Transition**: `q0, & -> q1`

3.2.3 Applications and Use Cases

NFAs are often used for pattern matching and text processing where multiple possibilities must be explored:

- **Regular Expressions**: Many regex engines convert patterns into NFAs (and subsequently DFAs) for efficient searching.
- **Pattern Recognition**: Finding specific subsequences within larger data streams using non-deterministic branching.

Practical Example: Suffix Matching This NFA (`examples/nfa/ends_with_aa_or_bb/`) demonstrates non-determinism by staying in the start state (`q0`) while simultaneously branching out to check if the current character starts the suffix "aa" or "bb".

```
[transitions]
q0, a -> q0, q1
q0, b -> q0, q3
q1, a -> q2
q3, b -> q4

[accept_states]
q2 q4
```

Listing 3: NFA matching strings ending in 'aa' or 'bb'

4 Pushdown Automata (PDA)

PDAs extend NFAs by adding a stack, allowing them to recognize context-free languages.

4.0.1 PDA Specific Sections

PDAs require slightly different alphabet definitions:

- `[alphabet_states]`: Symbols that can appear in the input string.
- `[alphabet_stack]`: Symbols that can be pushed onto the stack.

4.0.2 PDA Transition Syntax

Transitions include stack operations:

```
source_state, input_symbol, stack_pop -> next_state, stack_push
```

- `stack_pop`: The symbol required at the top of the stack. Use `&` to pop nothing.
- `stack_push`: Symbol(s) to push onto the stack. Use `&` to push nothing. Multiple symbols can be pushed (space-separated, pushed in reverse order).

4.0.3 Applications and Use Cases

PDAs are essential for recognizing context-free languages, which require memory beyond simple states:

- **Parsing**: Compilers use PDAs (often implemented as LL or LR parsers) to process nested structures like parentheses or mathematical expressions.
- **Markup Languages**: Verifying that tags in HTML or XML are correctly balanced and nested.

Practical Example: Balanced Sequences The classic language $L = \{0^n 1^n \mid n \geq 0\}$ (`examples/pda/zero_n_one_n/`) requires a stack to "remember" how many 0s were seen so they can be matched with an equal number of 1s. This is the foundation of parenthesis matching in programming languages.

```
[transitions]
# Push 0s onto stack
q0, 0, & -> q0, 0

# Pop a 0 for every 1
q0, 1, 0 -> q1, &
q1, 1, 0 -> q1, &

# Accept only if stack is empty (except for marker $)
q1, &, $ -> q_accept, &
```

Listing 4: PDA for matching balanced 0s and 1s

5 Context-Free Grammars (CFG)

CFGs provide a way to generate strings of a language using recursive production rules. In this project, CFG support has two distinct modes:

- **Generation**: produce one or more derivations from the grammar.
- **Verification**: determine whether a given input string can be derived from the grammar.

5.0.1 Required Sections

- `[variables]`: Space-separated list of non-terminal symbols.
- `[terminals]`: Space-separated list of terminal symbols.
- `[start_variable]`: The variable where derivations begin.
- `[rules]`: Derivation rules (productions).

5.0.2 CFG Rule Syntax

Rules follow the format:

```
Variable -> expansion1 | expansion2 | ...
```

Each expansion consists of space-separated variables and terminals. Use `&` for the empty string (epsilon).

5.0.3 CFG Execution Modes

- **Generation mode**: run the simulator with `CFG` and `--count <N>` without `--verify`. The emulator generates `N` derivations and prints each derivation step-by-step.
- **Verification mode**: provide an `input_string` together with `--verify`. The emulator checks whether the grammar can derive that string and reports the result as accepted or rejected.

Generated derivations are subject to a safety limit to prevent infinite expansion on recursive grammars. If a derivation is truncated, the output includes a warning so the result is not mistaken for a complete derivation. The limit can be adjusted from the command line with `--cfg-max-expansions <N>` in generation mode. Use `0` to disable the limit entirely.

5.0.4 Implementation Details

The emulator uses a memoized top-down parsing algorithm to determine if a string belongs to the language generated by the CFG. It automatically handles non-terminal expansions and backtracking, and it keeps track of active derivation states to avoid infinite recursion caused by left-recursive or epsilon-recursive grammars.

5.0.5 Applications and Use Cases

CFGs are used to describe the syntax of programming languages and natural languages.

- **Compiler Design**: Defining the syntax of high-level languages (C, Java, Python).
- **Linguistics**: Modeling the structural rules of human languages.
- **Data Transformation**: Defining formats for data serialization like JSON or YAML.

Practical Example: Nested Patterns The grammar for the language $L = \{a^n b^n \mid n \geq 1\}$ (`examples/cfg/an_bn/`) uses recursion to ensure that every 'a' is matched by a 'b'.

```
[rules]
S -> a S b | a b
```

Listing 5: CFG for $a^n b^n$

6 Automaton Transformation Tools

In addition to simulation, the project includes standalone file-to-file tools located in `tools/`. These utilities read and write the same ASL format used by the simulator, so the generated files can be fed back into the existing emulator without any additional conversion.

6.1 DFA / NFA Conversion

The `tools/convert_automaton.py` script supports deterministic and non-deterministic representations:

- `--to dfa`: Converts an NFA to an equivalent DFA using subset construction.
- `--to nfa`: Rewrites a DFA as an equivalent NFA-compatible ASL file.

```
python3 tools/convert_automaton.py --to dfa input.nfa output.dfa
python3 tools/convert_automaton.py --to nfa input.dfa output.nfa
```

6.2 DFA Minimisation

The `tools/minimise_dfa.py` script applies DFA minimisation and writes the minimized automaton back to disk using the same ASL structure.

```
python3 tools/minimise_dfa.py input.dfa output.min.dfa
```

6.3 Automaton Visualization

The `tools/visualize_automaton.py` script renders a diagram-only SVG preview of a DFA, NFA, or PDA using Graphviz `dot`. The output uses black transitions and labels on a white background for a clean academic presentation. It opens the preview in the default viewer, with platform-specific fallbacks on macOS, Linux, and Windows, and does not require a user-specified output file.

```
python3 tools/visualize_automaton.py examples/dfa/contains_11/contains_11.dfa
    DFA
python3 tools/visualize_automaton.py examples/pda/zero_n_one_n/zero_n_one_n.pda
     PDA
```